SQL & PostgreSQL — Detailed Study Guide

Expanded explanations, practical examples, and revision notes

1. Overview of SQL and Databases

What is a database?

A **database** is an organized collection of information stored electronically. It might hold customers, orders, products, grades, or sensor readings. The purpose of organization is to make data reliable to store, search, update, and analyze.

A **Database Management System (DBMS)** is the software that manages a database. It lets users define database structures, store and retrieve data, change records, control access, and recover from failures. A DBMS also helps coordinate concurrent activity so multiple users can work without corrupting the data.

SQL (Structured Query Language) is the language most commonly used to communicate with relational DBMSs. A user or application sends SQL commands; the DBMS checks and executes them against stored data. SQL is declarative: we describe the result or change we want, and the database chooses an execution plan.

SQL was developed at IBM in the 1970s (initially under the name SEQUEL) and later standardized. Today it is the common language of relational databases, though each product may provide extra features and slightly different syntax.

> **Key idea:** SQL is the bridge between people or applications and data managed by a DBMS. It lets us ask questions of the data and make controlled changes.

2. SQL Command Categories

Command categories describe the purpose of SQL statements. They are useful study groupings; terminology can vary a little between textbooks and database products.

Category	Full name	Typical commands	Purpose
DDL	Data Definition Language	`CREATE`, `ALTER`, `DROP`	Define or change database structures
DML	Data Manipulation Language	`INSERT`, `UPDATE`, `DELETE`	Add, change, or remove table rows
DQL	Data Query Language	`SELECT`	Read and return data
DCL	Data Control Language	`GRANT`, `REVOKE`	Manage privileges and access
TCL / transaction control	Transaction Control Language	`COMMIT`, `ROLLBACK`	Save or undo a transaction

A command's category tells you whether it changes the schema, changes records, reads data, changes permissions, or manages a transaction. In PostgreSQL, `BEGIN` (or `START TRANSACTION`) starts a transaction; `COMMIT` and `ROLLBACK` finish it.

3. Creating Tables with `CREATE TABLE`

A **table** stores records of one general subject. Each column has a name and a data type. Data types communicate what values are valid and how the DBMS stores and compares them.

```
CREATE TABLE table_name (
    column1 datatype [constraint],
    column2 datatype [constraint],
    ...,
    [table_constraint]
);
```

Common PostgreSQL types include:

- `INTEGER` / `INT` for whole numbers.
- `NUMERIC(precision, scale)` for exact decimal numbers, such as prices.
- `VARCHAR(n)` or `TEXT` for character strings.
- `DATE`, `TIME`, and `TIMESTAMP` for dates and times.
- `BOOLEAN` for true/false values.

Choose types that match the meaning of the data. A birthday should be a date, not an arbitrary text string; a count should generally be numeric. Type choices help prevent invalid values and enable correct comparisons.

Constraints: rules for valid data

Constraints are rules enforced by the database, protecting data even if it is entered through a different application.

- `NOT NULL`: a value must be supplied.
- `DEFAULT`: a value is supplied when an insert leaves the column out.
- `UNIQUE`: duplicate values are not allowed for the constrained value or combination.
- `CHECK`: values must satisfy a condition, e.g. `age >= 0`.
- `PRIMARY KEY`: identifies each row uniquely; primary key values cannot be null.
- `FOREIGN KEY`: requires a value to refer to a valid key in another table (subject to nullability and configured rules).

Example:

```
CREATE TABLE customer (
    cust_id     INTEGER PRIMARY KEY,
    first_name  VARCHAR(60) NOT NULL,
    last_name   VARCHAR(60),
    age         INTEGER CHECK (age >= 0),
    email_id    VARCHAR(255) UNIQUE,
    status      VARCHAR(20) DEFAULT 'active'
);
```

The `CHECK` prevents negative ages; `NOT NULL` requires a first name; `DEFAULT` fills in `active` when status is not supplied. A constraint can be written beside a column or, when it covers multiple columns, at table level.

> **Key idea:** Constraints are the table's "rules of the road." They preserve accuracy and consistency at the database level.

4. Primary Keys, Foreign Keys, and Relationships

A **primary key (PK)** identifies a row. No two rows can share the same primary key value, and a primary key cannot be `NULL`. A primary key can be one column (a simple key) or multiple columns (a composite key).

A **composite primary key** uses a combination of columns. For example, a student's enrollment might be uniquely identified by both student and course:

```
CREATE TABLE enrollment (
    student_id INTEGER,
    course_id  INTEGER,
    enrolled_on DATE,
    PRIMARY KEY (student_id, course_id)
);
```

A **foreign key (FK)** in one table refers to a primary or unique key in another table. This expresses a relationship and protects **referential integrity**: an order cannot point to a customer that does not exist.

```
CREATE TABLE customer (
    cust_id INTEGER PRIMARY KEY,
    first_name VARCHAR(60) NOT NULL
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customer(cust_id)
);
```

Here, each `orders.customer_id` must match a `customer.cust_id`. The source table example calls this `Customer_ID (FK → Customer.Cust_ID)`. A foreign key can be declared inline or as a table constraint. The database may be configured to reject deletion of a referenced customer or to take another action such as cascading; choose such behavior deliberately.

5. PostgreSQL Overview

PostgreSQL (often called Postgres) is an open-source object-relational DBMS. It supports standard relational features and many extensions. Common capabilities include transactions, foreign keys, subqueries, triggers, user-defined types and functions, and advanced query features.

Reasons teams choose PostgreSQL include its open-source license, strong ACID transaction support, extensive documentation, rich SQL functionality, and availability through major cloud providers. It is used for applications, reporting, analytics, and data warehousing.

PostgreSQL is not identical to every other SQL database. For example, commands such as `COPY`, `LIMIT`, and particular `ALTER COLUMN` forms are PostgreSQL-specific or not universal. Always consult the documentation for the DBMS in use.

6. Adding Data with `INSERT`

`INSERT` adds one or more rows. Prefer an explicit column list: it documents what each value means and avoids depending on the table's physical column order.

```
INSERT INTO customer (cust_id, first_name, last_name, age, email_id)
VALUES (1, 'Bee', 'Cee', 32, 'bc@example.com');
```

If no column list is supplied, values must be given for every column in the table's defined order (except columns with suitable defaults or generated values). This form is more fragile:

```
INSERT INTO customer
VALUES (1, 'Bee', 'Cee', 32, 'bc@example.com', 'active');
```

With a column list, the order of columns may be chosen by the writer; the values must follow that listed order:

```
INSERT INTO customer (cust_id, first_name, age, email_id)
VALUES (2, 'Dee', 22, 'd@example.com');
```

Omitted columns receive their default, `NULL` if permitted, or an error if required and no default is defined.

Insert multiple rows

```
INSERT INTO customer (cust_id, first_name, last_name, age, email_id)
VALUES
  (3, 'Eve', 'Fox', 35, 'eve@example.com'),
  (4, 'Gia', 'Hale', 42, 'gia@example.com');
```

Every value group must match the listed column count and compatible types. The original sample reused `cust_id = 1`; if `cust_id` is a primary key, the second insert with that ID would be rejected. Primary keys must be unique.

Bulk loading with PostgreSQL `COPY`

For large data imports, PostgreSQL's `COPY` can load a file efficiently:

```
COPY customer (cust_id, first_name, last_name, age, email_id)
FROM '/path/on/database-server/persons.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ',');
```

In PostgreSQL, server-side `COPY` reads a file accessible to the database server process, not necessarily your desktop. In `psql`, `\copy` is a client-side command that reads a file from the client computer. File paths and permissions matter. Validate CSV column order, headers, quoting, encoding, and data before loading.

7. Reading Data with `SELECT`

`SELECT` retrieves results without changing stored rows.

```
SELECT first_name, email_id
FROM customer;
```

Use `*` for every column:

```
SELECT * FROM customer;
```

For application queries, naming the required columns is often clearer and avoids fetching unnecessary data.

`DISTINCT`

`DISTINCT` removes duplicate result rows:

```
SELECT DISTINCT city
FROM customer;
```

With multiple selected columns, it removes duplicates of the entire combination, not of each column separately:

```
SELECT DISTINCT city, age
FROM customer;
```

This returns unique city-and-age pairs.

Aliases with `AS`

Aliases give output columns or tables temporary readable names:

```
SELECT cust_id AS "SerialNumber",
       first_name AS name
FROM customer;
```

In PostgreSQL, an unquoted alias is folded to lowercase; double quotes preserve the exact case and spelling. Quoted mixed-case identifiers must be quoted consistently later, so simple lowercase names are often easier to work with. Aliases do not rename the underlying database columns.

8. Filtering Rows with `WHERE`

`WHERE` keeps rows that satisfy a condition. It applies to individual rows before grouping.

```
SELECT first_name, age
FROM customer
WHERE age > 25;
```

Examples of conditions:

```
WHERE age = 25
WHERE age > 25
WHERE first_name = 'John'
```

Text literals use single quotes. Comparison operators include `=`, `<>` (not equal), `<`, `>`, `<=`, and `>=`.

Logical operators

```
WHERE age > 20 AND age < 30
WHERE age < 20 OR age > 30
WHERE NOT age = 25
```

`AND` requires both conditions; `OR` requires at least one; `NOT` reverses a condition. Parentheses make complex logic clear:

```
WHERE (city = 'Philadelphia' OR city = 'Seattle')
  AND age >= 18
```

Use parentheses rather than relying on memory of operator precedence.

`IN`

`IN` matches one of several values and can replace repeated `OR` conditions:

```
WHERE city IN ('Philadelphia', 'Seattle')
```

`BETWEEN`

`BETWEEN` checks an inclusive range: both endpoints count.

```
WHERE age BETWEEN 20 AND 30
```

This includes ages 20 and 30. It can also be used with dates and numeric values.

`LIKE` and wildcards

`LIKE` matches text patterns:

```
WHERE first_name LIKE 'Jo%'   -- begins with Jo
WHERE last_name LIKE '%son'   -- ends with son
WHERE name LIKE 'A_B'         -- A, any one character, then B
```

`%` matches zero or more characters; `_` matches exactly one character. Case sensitivity depends on database collation/operator. PostgreSQL also offers `ILIKE` for case-insensitive pattern matching.

> **Key idea:** Combine conditions carefully to describe the exact rows you need. Test a `SELECT` before using the same condition in a destructive statement.

9. Sorting and Limiting Results

`ORDER BY` determines presentation order. Without it, row order is not guaranteed.

```
SELECT * FROM customer
ORDER BY age DESC;
```

`ASC` sorts ascending and is the default; `DESC` sorts descending. Sort by multiple columns to break ties:

```
SELECT * FROM customer
ORDER BY city ASC, first_name DESC;
```

PostgreSQL's `LIMIT` caps the number of returned rows:

```
SELECT * FROM customer
WHERE age >= 25
ORDER BY age DESC
LIMIT 10;
```

The sort is important when selecting “top” rows; without it, the database is free to return any qualifying rows. For stable pagination, order by a unique tie-breaking column as well (for example, age and then cust_id).

10. Updating and Deleting Data Safely

`UPDATE`

`UPDATE` modifies values in rows matching a condition:

```
UPDATE customer
SET age = 17,
    last_name = 'Pe'
WHERE cust_id = 2;
```

The `SET` clause can assign constants or calculate from existing values. If `WHERE` is omitted, **every row** is updated.

`DELETE`

`DELETE` removes matching rows:

```
DELETE FROM customer
WHERE cust_id = 6;
```

```
DELETE FROM customer
WHERE age > 25;
```

Without `WHERE`, all rows are deleted, while the table structure remains. `DROP TABLE` is different: it removes the table itself.

Safety checklist: 1. Run a `SELECT` using the exact `WHERE` condition first. 2. Confirm the rows and expected count. 3. Use a transaction for important multi-step edits when appropriate. 4. Make sure backups and permissions are suitable. 5. Execute the `UPDATE` or `DELETE` only after checking.

11. Changing Table Structure with `ALTER TABLE`

`ALTER TABLE` evolves a schema without recreating the entire table. These examples use PostgreSQL syntax.

Add a column:

```
ALTER TABLE customer
ADD COLUMN phone VARCHAR(30);
```

Drop a column:

```
ALTER TABLE customer
DROP COLUMN phone;
```

Change a column's type:

```
ALTER TABLE customer
ALTER COLUMN age TYPE SMALLINT;
```

Rename a column:

```
ALTER TABLE customer
RENAME COLUMN first_name TO given_name;
```

Add a check constraint:

```
ALTER TABLE customer
ADD CONSTRAINT chk_customer_age CHECK (age >= 0);
```

Add a primary key:

```
ALTER TABLE customer
ADD PRIMARY KEY (cust_id);
```

Add a foreign key:

```
ALTER TABLE orders
ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_id)
REFERENCES customer (cust_id);
```

Before adding a constraint, existing rows must satisfy it; otherwise the command can fail. Dropping columns or changing types can affect applications and may discard or transform data. Plan migrations and check PostgreSQL documentation for locking and conversion behavior on large tables.

12. Aggregate Functions and Grouping

Aggregate functions summarize a set of rows:

Function	Meaning	Example
`COUNT(*)`	Counts rows	`SELECT COUNT(*) FROM sales;`
`COUNT(column)`	Counts non-NULL values in a column	`SELECT COUNT(order_id) FROM sales;`
`COUNT(DISTINCT column)`	Counts distinct non-NULL values	`SELECT COUNT(DISTINCT order_id) FROM sales;`
`SUM(expression)`	Totals numeric values	`SELECT SUM(profit) FROM orders;`
`AVG(expression)`	Calculates average	`SELECT AVG(age) FROM customer;`
`MIN(expression)` / `MAX(expression)`	Finds smallest/largest value	`SELECT MAX(sales) FROM orders;`

`COUNT(*)` includes rows even when some columns are `NULL`. `COUNT(column)` ignores rows where that column is `NULL`. Most other aggregates ignore null inputs; check the DBMS documentation for details.

`GROUP BY`

`GROUP BY` forms groups that share values in the specified column(s), allowing an aggregate for each group:

```
SELECT region, COUNT(customer_id) AS customer_count
FROM customer
GROUP BY region;
```

Here, the result has one row per region. Another example:

```
SELECT product_id, SUM(quantity) AS qty_sold
FROM sales
GROUP BY product_id
ORDER BY qty_sold DESC;
```

When selecting a non-aggregated column alongside aggregates, it generally must appear in `GROUP BY` (subject to DBMS rules).

`WHERE` versus `HAVING`

`WHERE` filters rows before grouping. `HAVING` filters groups after aggregates are calculated:

```
SELECT region, COUNT(*) AS customer_count
FROM customer
WHERE status = 'active'
GROUP BY region
HAVING COUNT(*) >= 10;
```

First only active customers are considered; then rows are grouped by region; finally groups with fewer than ten are removed.

> **Key idea:** Aggregates answer “how many?” and “how much?”; `GROUP BY` answers “per which category?”

13. String, Date/Time, and Numeric Functions

Functions transform values as part of a query. Availability and exact behavior can vary, so these examples are PostgreSQL-oriented.

String functions

- `UPPER(text)` / `LOWER(text)`: change letter case.
- `TRIM(text)`: remove leading and trailing spaces (or specified characters).
- `LTRIM(text)` / `RTRIM(text)`: trim the left or right side.
- `REPLACE(text, from, to)`: replace occurrences of text.
- `SUBSTRING(text FROM start FOR length)`: extract part of a string.
- `CONCAT(value1, value2, ...)`: concatenate values.
- `STRING_AGG(value, delimiter)`: combine values from multiple rows into a delimited string.

Examples:

```
SELECT UPPER(first_name) FROM customer;
SELECT TRIM('  hello  ');
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM customer;
```

Date and time

`CURRENT_DATE` returns the current date; `CURRENT_TIME` returns the current time. PostgreSQL also provides `CURRENT_TIMESTAMP` for date and time together.

```
SELECT CURRENT_DATE, CURRENT_TIME;
```

Use date/time types for date values and be mindful of time zones when storing or comparing timestamps.

Numeric helpers

`CEIL` (or `CEILING`) rounds a number upward to an integer; `FLOOR` rounds down. `RANDOM()` returns a pseudo-random value in PostgreSQL. `SETSEED()` sets the seed for PostgreSQL's random-number generator in a session.

```
SELECT CEIL(4.2), FLOOR(4.8), RANDOM();
```

These functions can be used in `SELECT`, and many can also appear in `WHERE` or `ORDER BY`. Applying a function to an indexed column in a filter can affect index usage; consider the query plan for performance-sensitive work.

14. Transactions and ACID

A transaction groups changes so they can be committed as a unit or rolled back. PostgreSQL example:

```
BEGIN;
UPDATE account SET balance = balance - 100 WHERE account_id = 1;
UPDATE account SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
```

If a step fails before commit, issue `ROLLBACK` to undo the transaction's uncommitted work. This is useful for tasks such as transferring money: both account changes should succeed together.

The **ACID** properties describe reliable transactions: - **Atomicity:** All steps happen, or none do. - **Consistency:** Rules and constraints remain satisfied. - **Isolation:** Concurrent transactions are managed so they do not interfere incorrectly. - **Durability:** Committed changes persist according to the DBMS guarantees.

Transaction behavior, isolation levels, and autocommit settings are important when multiple users change data concurrently.

15. Practical Query-Reading Order

A useful logical model for understanding a `SELECT` is: 1. `FROM` and joins identify source rows. 2. `WHERE` filters individual rows. 3. `GROUP BY` forms groups. 4. `HAVING` filters groups. 5. `SELECT` produces output expressions. 6. `DISTINCT` removes duplicate output rows. 7. `ORDER BY` sorts output. 8. `LIMIT` restricts the returned result.

This explains why aggregates generally cannot be tested in `WHERE`: grouping has not happened at that logical stage. Use `HAVING` for aggregate conditions. The DBMS may physically optimize execution differently, while preserving the logical result.

16. Integrated Example

Assume `customer(cust_id, first_name, city, age)` and `orders(order_id, customer_id, order_date, total)`.

Find adult customers in either of two cities:

```
SELECT cust_id, first_name, city
FROM customer
WHERE age >= 18
  AND city IN ('Philadelphia', 'Seattle')
ORDER BY city, first_name;
```

Show each order with the customer's name:

```
SELECT o.order_id, c.first_name, o.order_date, o.total
FROM orders AS o
JOIN customer AS c ON c.cust_id = o.customer_id
ORDER BY o.order_date DESC;
```

Calculate sales total by customer:

```
SELECT c.cust_id, c.first_name, SUM(o.total) AS total_spent
FROM customer AS c
JOIN orders AS o ON o.customer_id = c.cust_id
GROUP BY c.cust_id, c.first_name
HAVING SUM(o.total) > 500
ORDER BY total_spent DESC;
```

The final query joins related tables, groups each customer's orders, calculates a total, filters groups using `HAVING`, and sorts the result.

17. Common Mistakes and Good Habits

- **Forgetting a `WHERE` clause:** Can update every row or delete every row. Preview with `SELECT` first.
- **Leaving out a column list in `INSERT`:** Makes a query depend on table column order. Prefer explicit columns.
- **Reusing a primary key:** Primary-key values must be unique.
- **Expecting natural row order:** Add `ORDER BY` whenever order matters.
- **Using `= NULL`:** Use `IS NULL` or `IS NOT NULL`.
- **Confusing `WHERE` and `HAVING`:** Use `WHERE` for row conditions and `HAVING` for group/aggregate conditions.
- **Joining on the wrong columns:** This can create incorrect or excessive rows. Verify the `ON` condition.
- **Assuming all SQL products behave identically:** Check the relevant dialect, especially for imports, date functions, limits, and schema changes.

18. Final Takeaway and Revision Checklist

The core workflow is: define a table with `CREATE TABLE`, protect it with constraints, insert data with `INSERT`, read it with `SELECT`, narrow results with `WHERE`, and summarize them with aggregate functions and `GROUP BY`. Use keys to model relationships, joins to query related tables, `ALTER TABLE` to evolve the schema, and transactions to control groups of changes.

- SQL stands for **Structured Query Language**.

- DDL changes structure; DML changes records; DQL reads; DCL manages access; transaction commands save or undo work.
- PK uniquely identifies a row; FK references a key in a related table.
- `IN`, `BETWEEN`, and `LIKE` make common filters concise.
- `ORDER BY` controls output order; `LIMIT` restricts rows.
- `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX` summarize data.
- `GROUP BY` computes summaries per category; `HAVING` filters those groups.
- PostgreSQL supports standard SQL plus useful extensions; consult its documentation for exact syntax.